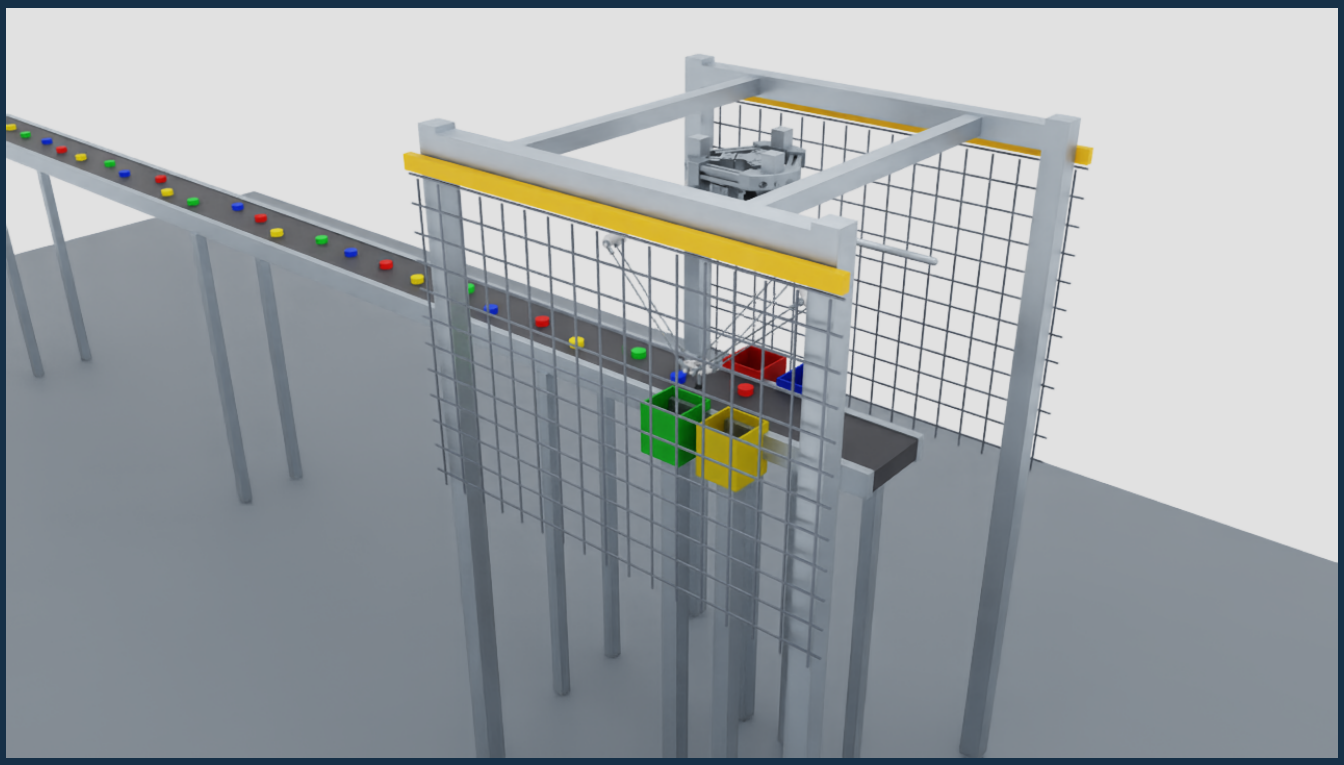
A PRACTICAL PROJECT BOOK

The Delta Robot Sorting Project

Understand the machine. Follow the ROS 2 control loop.
Build the next feature with confidence.



The actual workcell, rendered in Isaac Sim after the fastener correction.

NVIDIA Isaac Sim • ROS 2 Jazzy • RGB vision
Parallel robot motion • Stop-and-pick automation

WORKSPACE EDITION
24 September 2026

START HERE

A guide to your actual project

Simple explanations, real files, and an end-to-end view of the machine.

This project sorts loose plastic bottle caps into red, blue, green, and yellow bins. A simulated RGB camera sees the caps. ROS 2 interprets the image, chooses a target, plans the robot motion, and tells the conveyor and vacuum gripper what to do. Isaac Sim supplies the rendered scene and physical response.

The robot is built from the supplied **Delta+Robot** CAD assembly. Its three powered shoulders move a shared platform through six lower rods. The baseline workflow stops the belt before picking; the mechanism moves through controlled joint drives.

What this book will help you do

- Recognize the machine parts and the software responsible for each one.
- Follow a cap from camera pixels to a physical pick and a bin placement.
- Run and inspect the application, then choose a sensible first extension.

> **Scope of this edition**
> Prepared on 24 September 2026 from the existing workspace. The recorded simulation tests are prior project evidence, not new runs performed while writing this book. No existing project code, configuration, CAD, or scene asset was changed to create the book.

Choose a reading route

Your goal	Start with
Understand how it works	Pages 4–19: parts, ROS connections, perception, and motion.
Run it and diagnose a problem	Pages 20–25 and 32: recovery, startup, inspection, settings.
Build on the project	Pages 26–29 and 31: extension ideas and a testing sequence.

All example paths are relative to /home/amin24/orca/deltarobot unless a full path is shown. Distances in configuration and messages are metres; joint angles are radians.

CONTENTS

Find your way around

Each page is a short lesson. Click an entry to jump directly to it.

01 The workcell

Meet the sorting machine 4
Know the scale and layout 5
From CAD to a physical robot 6
How the Delta robot moves 7

02 The software and ROS 2

A tour of the project folders 8
ROS 2 in everyday language 9
The complete control flow 10
The interfaces you will actually use 11
The bridge between ROS and Isaac 12

03 Seeing the caps

How the camera finds a cap 13
Understand the coordinate frames 14
Turn a pixel into a pick point 15

04 Sorting and motion

The sorting state machine 16
Walk through one real red-cap pick 17
How ROS actually moves the joints 18
The conveyor and vacuum gripper 19
Failure, stop, and recovery 20

05 Using the project

Start the connected workcell 21
Watch the system without editing it 22
The configuration map 23
Tune color detection methodically 24
Change layout and motion carefully 25

06 Building on it

Choose your next extension 26
Example extension: a new cap color 27
Improve perception without rewiring everything 28
From simulation toward real hardware 29

07 Evidence and diagnosis

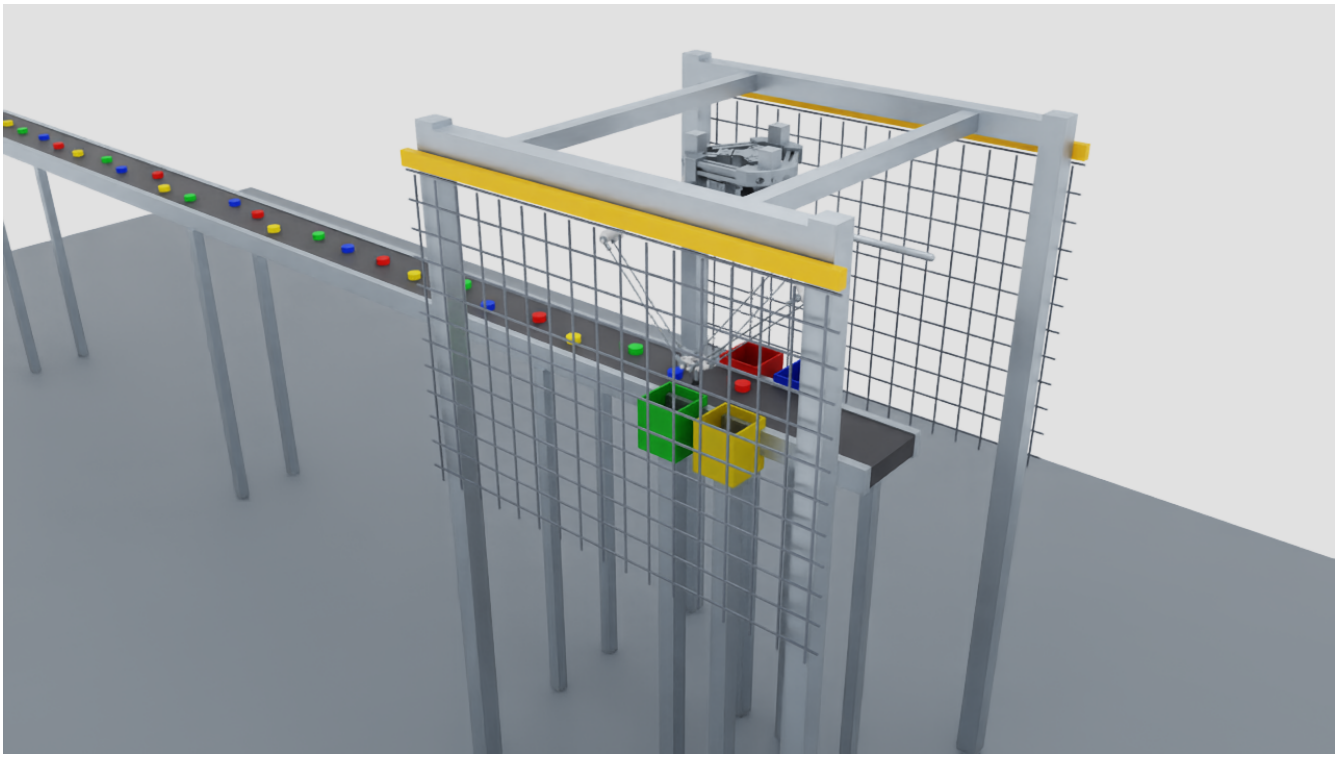
What has actually been validated 30
Test a future change in small steps 31
Find the first broken connection 32

Reference

A small working glossary 33
Where to look next 34

Meet the sorting machine

A small Delta robot inside a guarded conveyor workcell.



Actual Isaac Sim overview from the follow-up run after the moving-fastener correction. This is a saved simulation render, not an illustration of a proposed machine.

Follow the physical path

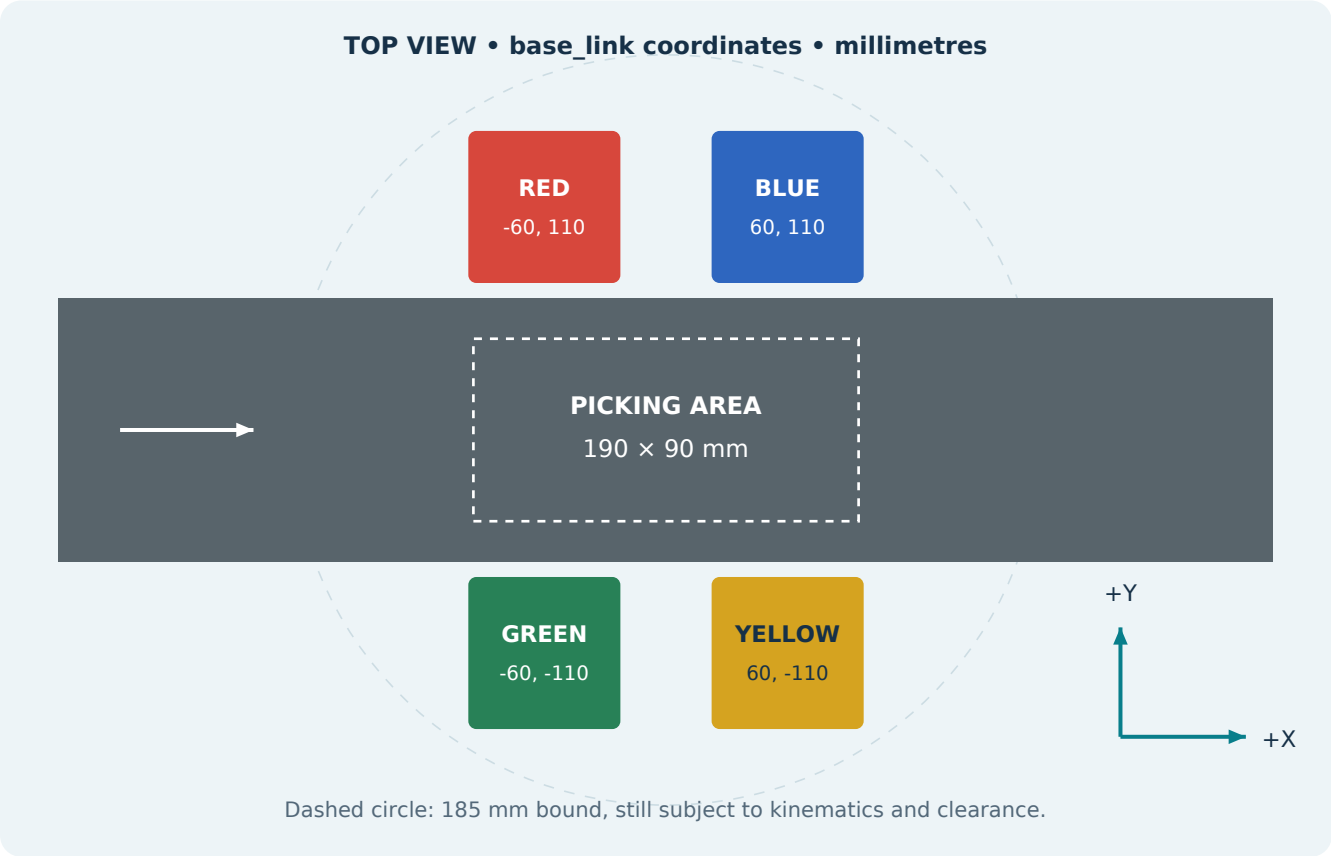
Caps enter along the long belt on the left. Near the robot, the camera observes a small picking area. The belt pauses, the vacuum cup lifts one cap, and the platform carries it to a nearby bin. The robot returns to a parked position that clears the camera view.

The rigid support frame carries the robot above the conveyor. Structural posts, floor, lights, and guards make the scene resemble a compact industrial station. They also give a clear sense of scale and keep the robot, belt, and containers in a coherent arrangement.

> **Why the belt stops**
> A stationary cap is easier to locate and grip reliably. The task deliberately waits for settling and then accepts a new image. Predicting a moving cap’s future position is a separate, unimplemented extension.

Know the scale and layout

The source robot has a compact reach, so the four bins sit close to the belt.



Schematic plan of the picking area. Bin labels give centre X, Y in millimetres. The conveyor continues well beyond this local view.

Part	Current size or location
Conveyor	3.4 m long; 130 mm wide; top 800 mm above the floor.
Caps	28 mm diameter, 12 mm high, 2.2 g each.
Robot and camera	Shoulder plane at 1.17 m; camera at 1.10 m, looking down.
Bins	75 mm inner width; rim at world Z = 0.820 m.

The camera can see more than the accepted picking rectangle. Projection rejects detections outside that rectangle, including cap-like colored regions in the bins. Being visible is only the first requirement; a target must also be reachable.

From CAD to a physical robot

The visual model and the moving mechanism serve different purposes.

The original folder contains one positioned STEP assembly and fourteen STL component meshes. It does not contain a ready-made ROS robot description, controller, or measured actuator limits. The conversion therefore reads the STEP assembly to recover how the supplied parts fit together.

Stage	What the project does
Read the assembly	Resolve 309 positioned CAD leaf instances, including repeated components.
Measure the mechanism	Extract shoulder axes and ball-joint centres; retain small assembly offsets.
Convert scale and axes	Change millimetres to metres and orient the hanging robot along world −Z.
Group moving bodies	Attach each visual component to its correct fixed base, upper arm, rod, or platform.
Add physical behavior	Create rigid bodies, joint drives, passive joints, and simple collision shapes.

Detailed appearance, simpler contact geometry

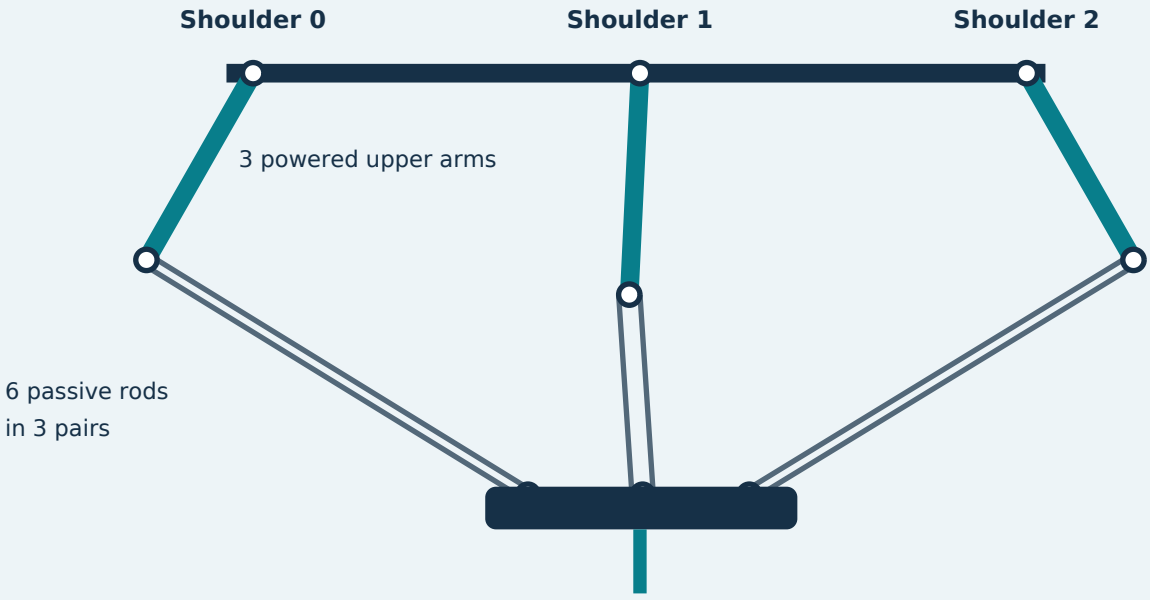
The visible CAD preserves the supplied machine’s shape. Capsules and a platform cylinder give physics simpler shapes to handle. A collision shape is the invisible surface used to decide whether bodies touch; it does not need every screw thread or gear tooth.

The generated USD scene contains the physical workcell. The original STEP and STL files remain unchanged. Conversion outputs live in **assets/generated/**; the conversion scripts live in **scripts/**.

> **A useful lesson from the floating component**
> Four fasteners previously belonged to the wrong moving bodies. They looked correct at rest and separated during movement. Their ownership is now corrected, and an audit checks the moving fasteners. When adding a visual part, verify its attachment during motion as well as at rest.

How the Delta robot moves

Three shoulder angles cooperate to position one shared platform.



Conceptual linkage diagram, not a CAD drawing. The real shoulders are spaced around the base; this flattened view shows how all three legs share one platform.

Each motor turns an upper arm. Two lower rods connect that arm to the moving platform. All three legs must agree on where the platform can be, so moving one motor affects the whole mechanism. The paired rods keep the platform approximately level.

This is a **parallel mechanism**: several paths connect the fixed base to the same moving body. The scene includes three driven revolute joints and twelve passive spherical joints, which act like ball joints at the rod ends. PhysX enforces those connections.

Term	Plain meaning
Inverse kinematics (IK)	Given a desired cup position, calculate the three shoulder angles.
Forward kinematics (FK)	Given the three shoulder angles, calculate the platform position.
Three degrees of freedom	The task commands X, Y, and Z translation. It does not command arbitrary wrist rotation.

Measured upper arms are about 178 mm long; lower rods are about 245 mm. The 5–85° shoulder limits are conservative assumptions, because the CAD does not specify measured hardware limits.

A tour of the project folders

Start with responsibilities; read the detailed implementation only when you need it.

Location	What you will find there
Delta+Robot/	Original STEP assembly and STL files. Keep these as the source model.
assets/generated/	Converted visuals, geometry measurements, and sorting_cell.usdc.
sim/scene.py	Builds the robot, conveyor, caps, bins, camera, frame, and physics scene.
sim/run_cell.py	Starts Isaac, steps physics, renders RGB, and runs the ROS adapter.
sim/ros_bridge.py	Translates ROS messages and service calls into sensors and physical actuators.
ros2_ws/src/delta_interfaces/	Definitions for detections, targets, and the MoveDelta action.
ros2_ws/src/delta_sorting/	The five application nodes, kinematics, vision, configuration, and launch file.
scripts/	Launch/build helpers, CAD conversion, scene audits, and validation tools.
tests/	Focused tests for perception, kinematics, and path checks.
artifacts/ and docs/	Saved runs and evidence; existing technical documentation and this book.

Three useful starting files

Read **coordinator_node.py** to understand the sequence of operations. Read **motion_node.py** to see how a movement request becomes joint commands. Read **sim/ros_bridge.py** to see exactly how those commands reach Isaac.

> **Opening the scene is not the complete application**
> The USD file stores the workcell. Autonomous sorting also needs the external ROS nodes and the running adapter. Use scripts/run_demo.sh to start the connected system.

ROS 2 in everyday language

ROS 2 lets specialized programs exchange information and requests.

ROS term	Think of it as…	Example here
Node	A small program with one job.	cap_perception finds colored caps.
Topic	A named stream of messages.	/camera/image_raw carries images.
Message	The agreed format of one item.	An Image includes pixels, size, and timestamp.
Service	A short request and reply.	Stop the conveyor; receive success or failure.
Action	A goal that takes time and reports progress.	Move the cup; report error and final result.
TF	A directory of coordinate relationships.	Where the camera is relative to the robot.
Simulation clock	Time measured by the virtual world.	The settling delay follows /clock.

Why several programs?

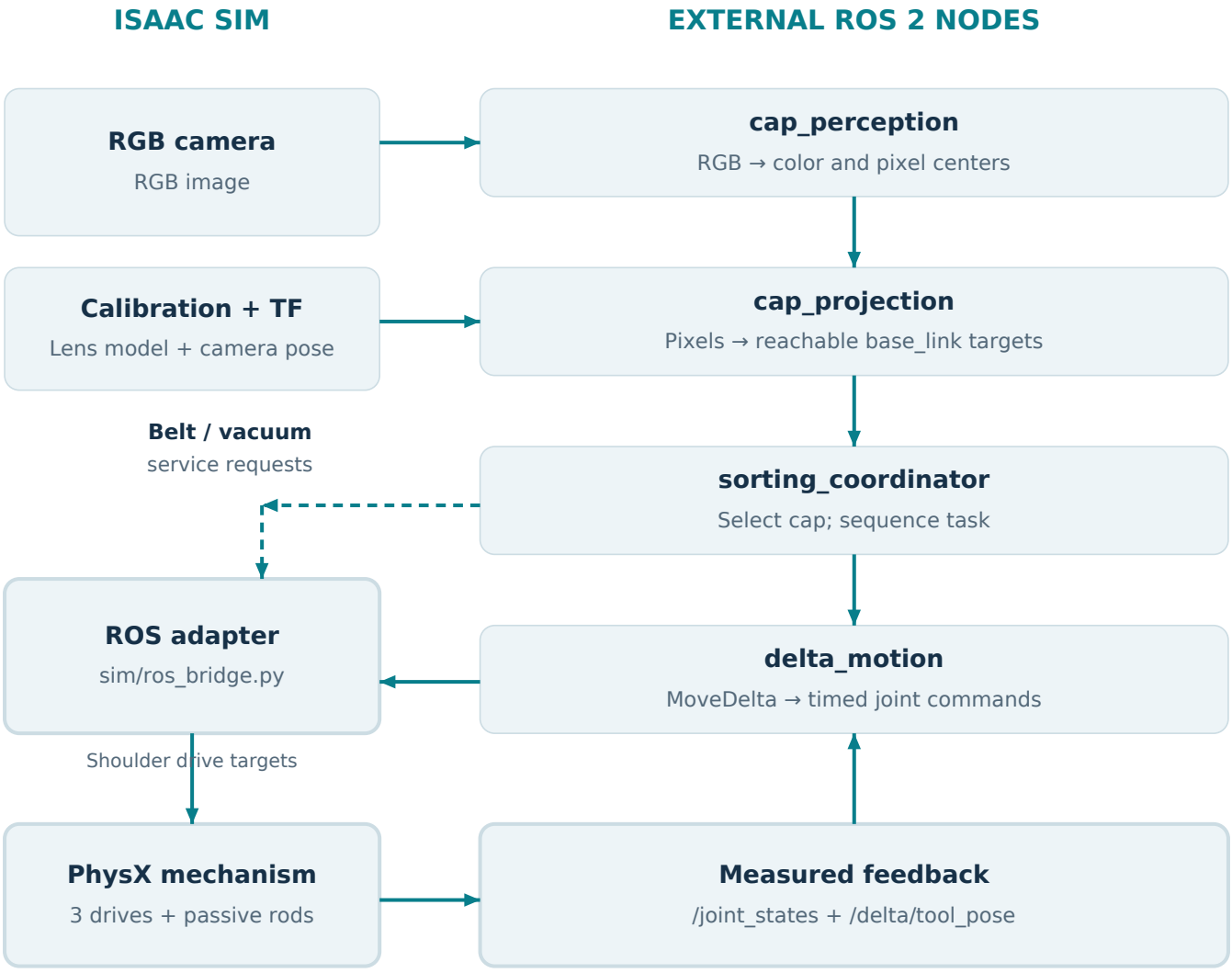
Vision can change without rewriting motor control. A motion failure can be diagnosed without guessing whether the color detector was wrong. Each boundary carries a small, explicit result: a pixel detection, a physical target, a movement goal, or a measured state.

The five external nodes are **cap_perception**, **cap_projection**, **delta_motion**, **sorting_coordinator**, and **sorting_evaluation**. A sixth node, **isaac_cell_bridge**, runs inside the Isaac process as its adapter.

> **How they find each other**
> The launcher uses ROS_DOMAIN_ID=42 by default and Fast DDS through rmw_fastrtps_cpp. An observation terminal must use the same domain and source both Jazzy and this workspace. You do not need to write your own network protocol.

The complete control flow

Images go out of Isaac; joint commands come back in; measured motion closes the loop.



Solid arrows show the main data and control path. Dashed arrows are short conveyor/vacuum requests. Evaluation observes separately and never supplies picking targets.

The coordinator chooses *what to do next*. The motion node decides *how to get there*. The adapter applies *physical commands*. Those three roles are deliberately separate, so ROS 2 controls the actual mechanism throughout the sorting cycle.

The implementation enables isaacsim.ros2.bridge and uses an in-process Python rclpy adapter. Its sorting logic is not an OmniGraph graph embedded in the USD scene.

The interfaces you will actually use

These are the connections between the boxes on the previous page.

Interface	Content and purpose
/camera/image_raw	sensor_msgs/Image: rendered RGB8 pixels.
/camera/camera_info	sensor_msgs/CameraInfo: image size, lens calibration, and distortion values.
/caps/pixels	delta_interfaces/PixelCaps: pixel centres, pixel radii, color, confidence, and rejection count.
/caps/targets	delta_interfaces/CapTargets: 3D positions in base_link, color, confidence, rejection count.
/delta/move_to	MoveDelta action: cup target in metres plus speed in m/s; returns success/message and position-error feedback.
/delta/joint_command	sensor_msgs/JointState: commanded shoulder_0, shoulder_1, shoulder_2 positions in radians.
/joint_states; /delta/tool_pose	Measured joint state and measured cup PoseStamped, used by motion control.
/conveyor/set_running	SetBool service: true starts the belt; false stops it.
/gripper/set_enabled	SetBool service: true requests attachment; false releases.
/sorting/state; /sorting/events	Readable state name and JSON task events for observation and reports.
/tf; /tf_static; /clock	Dynamic/static coordinate transforms and simulation time.

Feedback also includes /conveyor/running, /gripper/attached, and the latched /cell/estopped state. Stop/reset services are explained on page 20. /evaluation/ground_truth and /evaluation/done belong to evaluation.

The bridge between ROS and Isaac

The adapter is the machine’s sensor and actuator interface.

On the way out: publish what the simulator measures

The RGB render becomes an Image message. The lens settings become CameraInfo. The adapter publishes measured shoulder angles, the measured vacuum contact point, coordinate transforms, and the simulation clock. The other nodes receive these through ordinary ROS subscriptions.

On the way in: apply physical commands

The motion node sends three shoulder angles. The adapter checks the names, number of values, timestamp, finite numbers, and joint limits. Valid commands become targets for the three USD revolute-joint drives. It converts radians into the degree convention used by those drives and accounts for the CAD reference angle.

The adapter also implements the belt and vacuum services. It contains no HSV detector, bin-selection policy, or Delta inverse kinematics. Those decisions belong to the external ROS application.

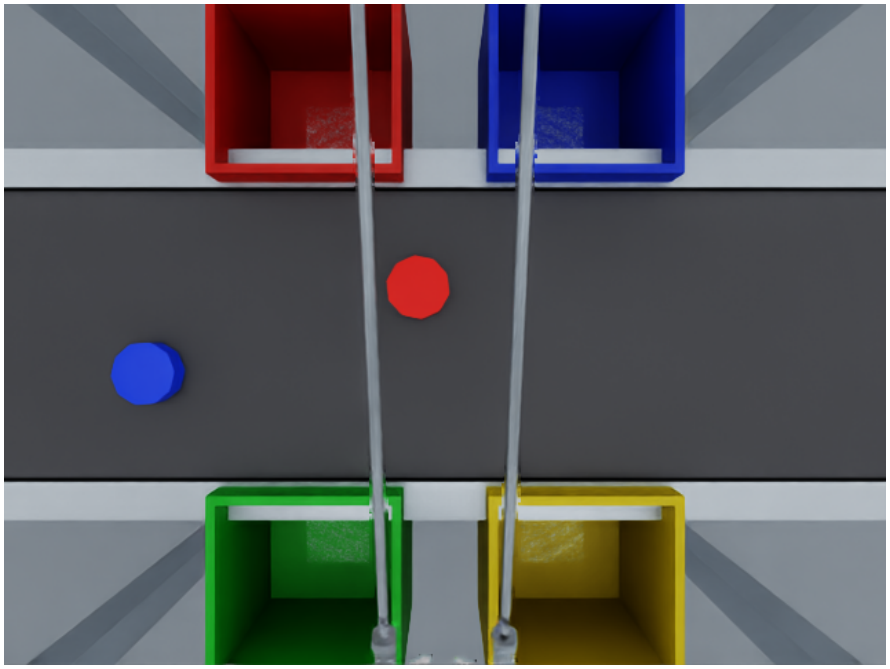
Stream or process	Default rate
Physics solver	240 steps per simulation second.
Robot state and clock publication	60 updates per simulation second.
Motion command timer	50 updates per simulation second.
RGB camera	10 frames per simulation second.

> **Simulation time versus wall time**
> All application nodes use use_sim_time=true. If rendering runs slower than real time, a simulated second takes longer on your clock. The motion controller also has a wall-time watchdog to catch stalled simulator feedback.

Robot and grasp geometry are measured inside physics to emulate actuator feedback and attachment. Cap identity and true color are sent only on the separate evaluation channel; they do not determine visual targets.

How the camera finds a cap

The detector works with pixels, not the simulator’s object names.



Actual overhead RGB image. The bins are visible, but only candidates inside the belt picking region can become targets. Some linkage rods remain in view.

OpenCV converts the image from RGB to **HSV**. Hue describes the color family; saturation helps separate vivid colors from gray; value describes brightness. Configured ranges make a mask for red, blue, green, and yellow.

The detector cleans small noise and looks at each blob’s outline. A plausible cap must have suitable area, roundness, solidity, and color purity. A clipped edge, overlapping object, or nearby colored fragment can cause rejection. The surviving outline gives a centre and an approximate radius in pixels.

> **What confidence means**
> The published score summarizes shape quality and color purity. It is not a calibrated probability. A confidence of 0.90 does not mean a measured 90% chance that a future pick will succeed.

Implementation: vision.py performs the image processing; perception_node.py receives images and publishes /caps/pixels. Gray, purple, overlapping, split-color, and border-cropped cases are covered by existing tests.

Understand the coordinate frames

A coordinate only makes sense when you know where its origin and axes are.

Frame	How to picture it
world	The factory floor is Z=0. Positive X follows conveyor travel; positive Z points upward.
base_link	The origin is at the robot’s shoulder-centre plane, 1.17 m above the floor. Axes match world.
conveyor	The origin lies on the belt top at world Z=0.80 m.
camera_optical	X points right in the image, Y down, and Z forward along the camera’s view. Here forward points down toward the belt.
tool0	The actual vacuum contact point on the moving platform, including the cup offset.
upper_0 … upper_2	Physical upper-arm frames at the shoulder pivots.

Why are the target Z values negative?

The robot origin is above the conveyor. A cap top at world height 0.812 m is **0.358 m below the shoulders**. Its Z coordinate in base_link is therefore −0.358 m. This describes a point well above the floor, not a point under it.

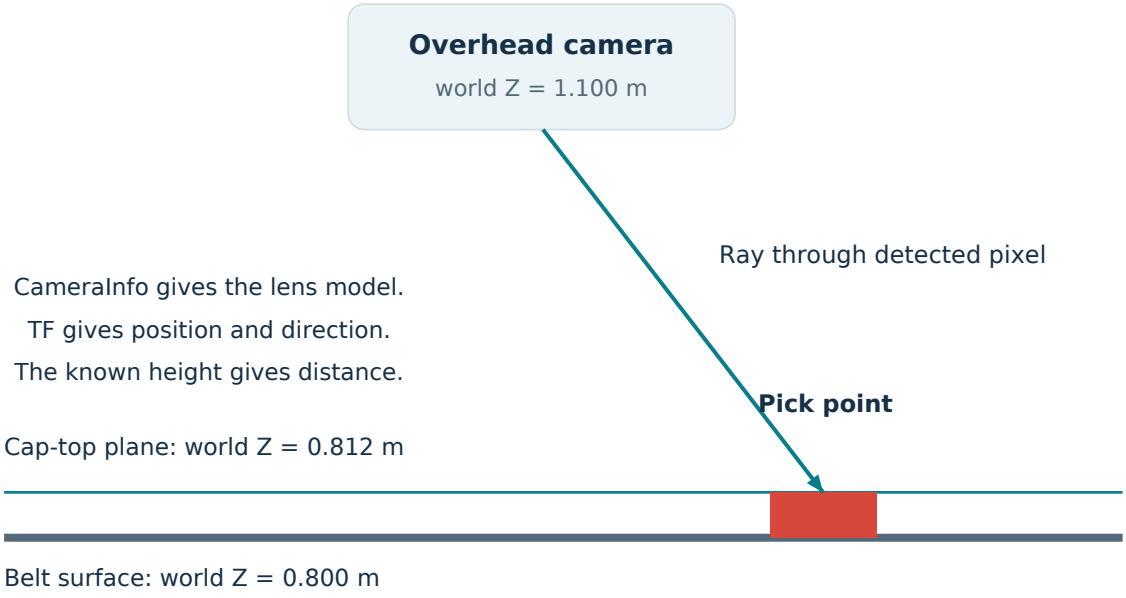
```
world cap-top Z = 0.800 + 0.012 = 0.812 m
base_link Z    = 0.812 - 1.170 = -0.358 m
```

TF stores these relationships. The projection node asks TF for the camera pose at the image timestamp, then expresses the result in base_link. The coordinator and motion action use that same frame, so a target does not silently change meaning between nodes.

> **A common mistake to avoid later**
> Bin X/Y and motion target Z are in base_link. Belt top, bin floor/rim, and camera position are configured in world coordinates. Read the field’s meaning before copying a value from one place to another.

Turn a pixel into a pick point

The camera supplies a direction; the known cap height supplies the missing distance.



This baseline assumes an upright cap on a flat belt. It intersects the viewing ray with the cap-top plane rather than pretending a single RGB pixel measures depth.

Imagine drawing a line from the camera through the detected centre of a cap. CameraInfo tells the software which direction that pixel represents. TF places the line in the robot's frame. The point where it crosses the known cap-top plane is the estimated picking position.

For the current 640 × 480 image, the centre is (320, 240). The pinhole focal scale is 480 pixels in both directions. The simulation has zero distortion. The projection node rejects nonzero distortion values, so a future real camera must provide rectified images or an updated projection method.

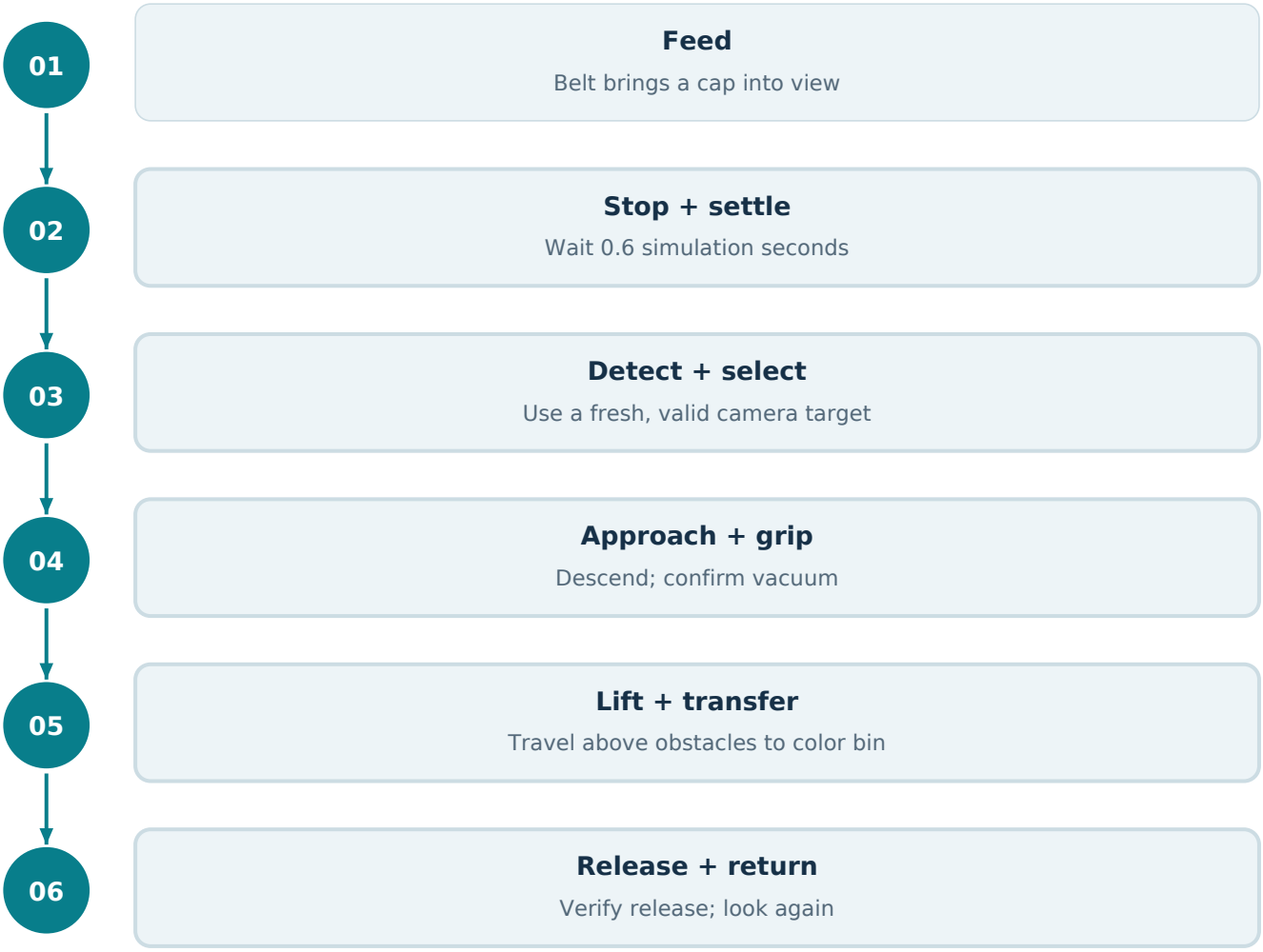
Checks before a target is published

- The ray must point toward the plane and intersect it sensibly.
- The projected cap radius must lie between 10 and 19 mm.
- The centre must lie inside the picking rectangle and pass inverse kinematics.

After selection, the commanded cup point is 1 mm above the estimated cap top to suit the ideal vacuum model. Unknown object heights, hidden stacked caps, and arbitrary cap orientations are outside this calibrated baseline.

The sorting state machine

A state machine is a sequence that waits for evidence before taking the next step.



Simplified normal cycle. The implementation uses additional states to wait for service replies, action results, release confirmation, and recovery.

At startup, the coordinator waits for the ROS interfaces and simulation clock, stops the belt, releases the empty cup, and moves home. It then starts feeding caps. Once a valid candidate is visible, it stops the belt and waits for settling.

Detection continues in the background, but picking accepts only a fresh frame taken after the settling delay. The selected cap must have a recognized bin color, enough confidence, and a feasible approach. The coordinator favors the candidate with the largest X, closest to leaving the picking region.

> **One cap does not always mean one belt restart**
> After placing a cap, the robot returns home and looks again while the belt remains stopped. It can pick another valid cap already in the area. If none is found within the detection timeout, it resumes feeding.

Walk through one real red-cap pick

Use one recorded attempt to connect the messages to physical movement.

In the first attempt of the saved 20-cap run, the visual target was approximately **X = −5.8 mm, Y = +20.9 mm, Z = −358 mm** in base_link. The color detector classified it as red. No simulator object ID was used to choose that point.

Step	What the coordinator requests
1. Approach	Move the cup above the detected X/Y at travel_z = −0.295 m.
2. Descend	Move slowly to Z = −0.357 m, 1 mm above the estimated cap top.
3. Grip	Call /gripper/set_enabled with true; wait for a successful attachment reply.
4. Lift	Return vertically to travel height, then confirm the cap is still attached.
5. Transfer	Move at travel height to the red-bin centre: X = −0.060 m, Y = +0.110 m.
6. Release	Descend to release_z = −0.305 m; disable vacuum and verify detachment.
7. Return	Move to home [0, −0.125, −0.300] and look for the next cap.

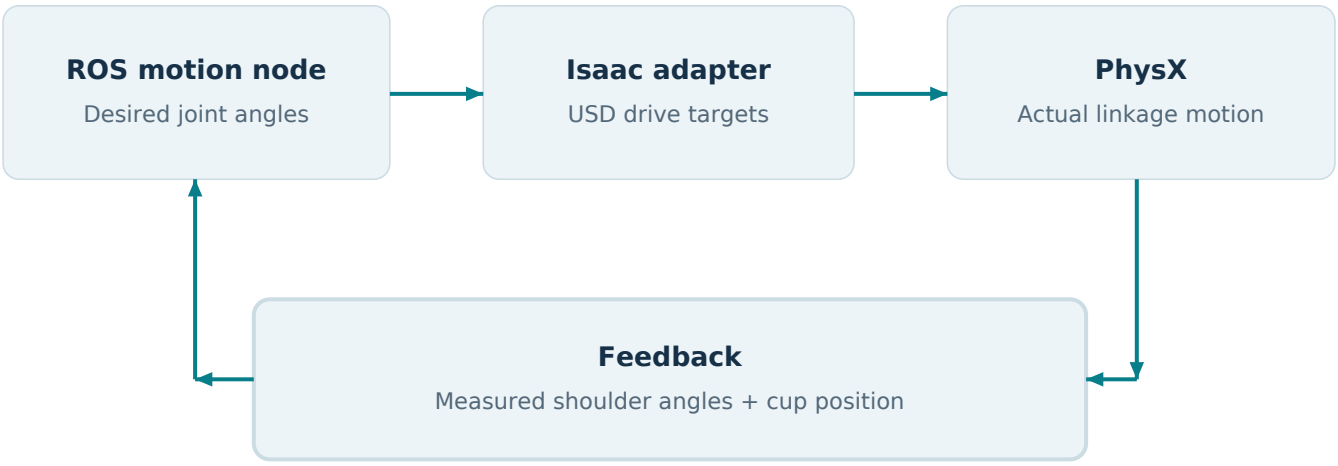
How the result was checked

The evaluator observed the held cap, confirmed that it rose above the belt, and checked its final position inside the red bin. The reported first placement settled at approximately world [−0.0602, +0.1085, +0.7360] m. That final Z uses the floor-based world frame, unlike the action targets above.

Evidence: artifacts/twenty_attempts/results.json, attempt 1. These numbers describe a recorded simulation result, not a promised physical accuracy for a real robot.

How ROS actually moves the joints

A movement goal is translated into many small commands, then checked against feedback.



The action succeeds after the measured robot reaches the goal.

The physical platform follows the constrained linkage. The controller does not set a new platform transform for every waypoint.

The coordinator sends a **MoveDelta** action goal containing the desired cup position and speed. The motion node first checks the entire proposed straight segment. It samples intermediate points, solves inverse kinematics at each one, and rejects paths that violate the configured limits or clearance checks.

A smooth timing profile starts and ends gently. If the planned joint speeds or accelerations would be too high, the node lengthens the motion. It publishes the resulting three-angle commands at 50 Hz in simulation time.

Isaac’s joint drives then move the physical upper arms. The lower rods and shared platform follow because the joints constrain them. The motion node reads the actual shoulder angles and cup position; an accepted action is only a permission to start, not proof that the goal has been reached.

> **Completion is measured**
> The current position tolerance is 2 mm, with an additional joint-tracking and settling check. A stale feedback stream, cancellation, stop condition, or tracking timeout makes the action fail rather than silently count as success.

The planner uses conservative belt/bin tool envelopes. Robot self-collision pairs are filtered in physics. Neither feature is a general collision planner for arbitrary new layouts.

The conveyor and vacuum gripper

These are ROS-controlled physical mechanisms with simple simulation models.

Conveyor: motion through contact

The belt is a kinematic collider with a PhysX surface velocity. Friction makes free caps move along it. Setting the running service to false changes the surface velocity to zero; the task then allows 0.6 simulation seconds for settling. The adapter does not move caps by rewriting their positions.

The default belt speed is 0.065 m/s. Caps spawn with varied lateral positions, yaw, and a small initial tilt. Their collision model is a solid cylinder with friction 0.45, rather than a detailed hollow shell.

Vacuum: a physical attachment after a seal check

When ROS requests vacuum, the adapter checks that the belt is stopped and that a suitably upright cap top is close to the cup. The current tolerances are 7 mm sideways and 4 mm vertically. These are attachment-model tolerances, not measured camera accuracy.

If a cap qualifies, the simulator creates a breakable fixed joint between the cap and platform. Its local attachment frames preserve their measured relative positions, so attachment does not snap the cap into a new place. Releasing vacuum removes the joint; contact and gravity determine the fall into the bin.

Useful feedback	What it tells the task
/gripper/attached	Whether an attachment is currently held.
/conveyor/running	Whether the simulated belt is running.
Service success/message	Whether the requested operation was accepted and why it failed if not.

Where this model stops
There is no pressure, leakage, flexible seal, or suction-flow model. For a future physical gripper, replace the attachment decision with real actuator control and a reliable seal or pressure signal.

Failure, stop, and recovery

The task has explicit responses when a normal cycle cannot continue.

Situation	Current response
No valid target	Resume feeding after the detection timeout; eventually finish after the empty-feed timeout.
Uncertain or overlapping cap	Reject it before creating a movement goal.
Unreachable point or bad path	Reject the motion action before commanding the new trajectory.
Vacuum fails	Retry once; if still empty, release, lift vertically, return home, and feed onward.
Failure while holding a cap	Latch a fault, stop the belt, and retain the vacuum attachment.
Missing commands or feedback	The adapter holds its last target; the motion controller has stale-feedback and wall-time checks.

Operator stop and restart

Use a terminal prepared as on page 22. Stop through the coordinator so that the task and physical adapter agree about the fault state:

```
ros2 service call /sorting/stop std_srvs/srv/Trigger '{}'
```

With the cup empty, clear the physical stop first, then restart the task:

```
ros2 service call /cell/reset std_srvs/srv/Trigger '{}'
ros2 service call /sorting/reset std_srvs/srv/Trigger '{}'
```

A stop retains a held cap. Inspect its physical location before releasing it, because vacuum-off drops it there. Reset preserves existing cap positions and the attempt counter. Restarting the application creates a fresh cap supply and new task state.

This is a simulation interlock, not a certified industrial safety controller. /cell/estop accepts true to latch a stop; false does not clear it. Use the reset service.

Start the connected workcell

These commands match the installed workspace and its launcher.

Installed baseline	Recorded environment
Operating system and ROS	Ubuntu 24.04; ROS 2 Jazzy; system Python 3.12.
Isaac Sim	6.0.0-rc.59 at /home/amin24/isaacsim; bundled Python 3.12.
GPU used for validation	NVIDIA RTX 5070 Ti, 16 GB VRAM.

Start with the graphical view

```
cd /home/amin24/orca/deltarobot
./scripts/build.sh
./scripts/run_demo.sh
```

The build prepares the ROS packages. The launcher sources ROS and the workspace, starts the five external nodes, then starts Isaac with the adapter. The task automatically begins after the clock and interfaces are ready. Ctrl+C stops the processes started by the launcher.

Run a new 20-attempt headless session

```
DELTA_ATTEMPTS=20 DELTA_OUTPUT="$PWD/artifacts/my_run" \
  ./scripts/run_demo.sh --headless --max-time 700 \
  --exit-on-complete
cat artifacts/my_run/results.json
```

Use a new output directory for each experiment. Reusing a directory replaces its reports. The time limit is in simulation seconds. Without --exit-on-complete, the application remains open until its time limit, window closure, or interruption.

> **Keep the Python environments separate**
> Use system ROS Python for the ROS workspace and Isaac’s python.sh for the simulator. The project .venv is for offline CAD conversion. Do not activate it before launching ROS or Isaac. Generated assets are already available for ordinary runs.

On another machine, match the recorded Isaac/Jazzy combination or validate compatibility. README.md lists ROS dependencies and CAD rebuild commands. Set ISAAC_SIM_PATH if Isaac is installed in a different directory.

Watch the system without editing it

The quickest diagnosis is to find the first missing step in the data flow.

Open a second terminal while the simulation is running. Prepare that terminal once:

```
source /opt/ros/jazzy/setup.bash
source /home/amin24/orca/deltarobot/ros2_ws/install/setup.bash
export ROS_DOMAIN_ID=42
```

Run one observation command at a time; use Ctrl+C to stop an echo before starting the next. If you launched the cell on another ROS domain, use that domain here too.

Command	What to look for
ros2 node list	The five application nodes plus isaac_cell_bridge.
ros2 topic echo /sorting/state	States progressing through feeding, approach, grip, and return.
ros2 topic hz /camera/image_raw	A live image stream. Wall-clock rate can be below 10 Hz when simulation is slower than real time.
ros2 topic echo /caps/pixels	Image detections: color, pixel centre, and confidence.
ros2 topic echo /caps/targets	Accepted positions in base_link.
ros2 topic echo /joint_states	Measured shoulder positions while the robot moves.
ros2 topic echo /gripper/attached	A true/false change when the cup attaches and releases.

Use the saved evidence

The run folder contains **ros.log**, **isaac.log**, **events.jsonl**, **results.json**, **simulation.json**, and sampled camera PNGs. The event sequence shows where a cycle stopped. The results file reports measured outcomes; it should agree with what you see in the images and scene.

Add --capture-overview to a future launch to save overview snapshots as well. An image viewer must support the camera’s sensor-data QoS, which favors current frames over waiting for retransmission.

The configuration map

Most everyday tuning starts in one YAML file shared by the simulator and ROS nodes.

Default file: ros2_ws/src/delta_sorting/config/cell.yaml

Section / setting	Meaning and current baseline
conveyor.speed	Belt surface speed: 0.065 m/s.
caps.count, spacing, seed	24 caps; 0.12 m spacing; seed 17 for repeatable spawn variation.
caps.colors	Rendered cap colors as RGB values between 0 and 1.
camera	World position, 640×480 resolution, optics, and 10 Hz simulated frame rate.
perception	HSV bands, size/shape filters, and minimum confidence 0.80.
bins and bin_geometry	Color-to-centre mapping, container dimensions, and tool release height.
robot	Home/travel positions, working envelope, speed limits, tracking tolerance, and joint drive settings.
task	Autostart, settling, detection/operation timeouts, retry count, and empty-feed timeout.
simulation.physics_dt	Physics step: 1/240 s. Solver iteration settings are nearby.

A controlled way to experiment later

Copy the YAML to a new file, change one idea at a time, and pass its path as **DELTA_CONFIG**. Restart the launcher so every process reads the same settings. Keep the original configuration and a known-good run for comparison. Changes described here are future reader actions; this book did not apply them.

> **Two launcher overrides to remember**
> Use DELTA_ATTEMPTS for the attempt limit: the launcher supplies it separately and defaults to 24. Use --max-time for the run duration. Changing only task.max_attempts or simulation.max_time in a YAML copy will not override those launcher defaults. The current runtime also does not use simulation.render_dt; camera.frame_rate controls RGB cadence.

Tune color detection methodically

Look at the saved image before changing a threshold.

Color	Default OpenCV hue band
Red	0–10 and 170–179, because red wraps around the hue scale.
Blue	100–135.
Green	40–85.
Yellow	20–37.

A practical tuning sequence

1. Check the view. Confirm that the cap is on the belt, in the picking region, and not hidden behind the platform or another cap. The robot’s parked home position is part of the perception design.

2. Check the color mask. Hue selects the color family. The saturation minimum of 95 rejects washed-out colors; the brightness minimum of 55 rejects dark pixels. Adjust using actual rendered images and keep neighboring color bands distinct.

3. Check the shape filters. The baseline accepts areas from 300 to 3,200 pixels, with minimum circularity 0.77 and solidity 0.93. A larger camera distance or wider view makes caps smaller, so optics and area thresholds are connected.

4. Compare detections with targets. If /caps/pixels has a cap but /caps/targets is empty, investigate calibration, picking-region bounds, physical radius, and reachability before loosening color thresholds.

5. Retest difficult images. A threshold change that finds one missing cap might also accept a bin edge or touching objects. Keep empty, gray, overlap, border, and split-color examples in the test set.

> **Rendered RGB is not an HSV threshold**
> caps.colors controls material appearance. Lighting and rendering affect the pixels. Choose hue ranges from the camera image; do not assume the three material RGB numbers are the detector’s input.

Change layout and motion carefully

Reachability belongs to the whole route, not just the destination.

The current tool envelope uses a 185 mm radial bound and Z between −0.385 and −0.280 m in base_link. These bounds are only an initial filter. A target inside them can still fail the Delta geometry, assumed joint limits, or fixed-cell clearance checks.

Change you want	What must stay consistent
Move a bin	New X/Y, bin dimensions, rim height, release height, transfer route, and return route.
Change cap height or belt height	The cap-top projection plane, picking descent, physical scene, and camera view.
Move the camera vertically or sideways	Its world position, TF, visible picking region, and apparent cap pixel size.
Tilt the camera	The current scene/TF assume a downward-facing camera. This needs a future scene and transform change, not just a position edit.
Change the tool length	The measured geometry/tool offset, physical cup, tool feedback, and IK model must agree.

Speed settings have different jobs

cartesian_speed = 0.12 m/s limits ordinary cup travel. **approach_speed = 0.045 m/s** slows descent and lifting. **joint_speed = 0.85 rad/s** and **joint_acceleration = 2.0 rad/s²** protect the commanded shoulder trajectory from excessive rates. Time scaling can make a move slower than the requested Cartesian speed.

Drive stiffness, damping, and torque limits describe how the simulated actuators respond. They are not simple speed knobs, and the CAD did not supply validated motor data. Establish stable motion and physical tracking before changing them.

> **A useful boundary**
> An earlier wider bin arrangement was rejected by IK. Expanding workspace_radius alone cannot lengthen the real links or remove joint limits. For substantial layout changes, add broader collision checking and validate complete pick-to-bin routes.

Choose your next extension

Add one capability at a time, keeping a working sorting cycle as your reference.

Extension	Where to connect it	A useful first milestone
Read-only dashboard	Subscribe to /sorting/state, /sorting/events, and gripper/belt feedback.	Show current state, attempt count, and last failure without sending commands.
Broader cap populations	Configurable cap seed, count, spacing, and colors.	Compare several saved runs, each with its configuration and report.
Another color or bin	caps.colors, hue_ranges, bins, and route validation.	Demonstrate one correctly classified and placed new color.
Improved detector	Replace or extend the image-processing node while preserving its output interface.	Publish valid PixelCaps on saved and live images.
Depth or varied heights	Add sensing and adapt projection/target generation.	Estimate an elevated cap correctly without the fixed top plane.
Physical robot adapter	Replace the simulated actuator/sensor boundary with hardware interfaces.	Read real joint positions and validate one restricted movement.
Moving-belt picking	Tracking, belt motion estimates, and interception planning.	Predict where one moving cap will be when the tool arrives.

A good first software addition

A read-only dashboard or run analyzer is a small, useful extension. It can observe the current interfaces while leaving sorting decisions with the coordinator. To report confirmed placements, use the evaluator’s result files; counting release requests alone can overstate success.

> **Keep one owner of robot commands**
> A future dashboard should not publish joint commands while the existing motion node is also controlling the robot. Manual control needs an explicit mode that gives one controller ownership and disables autonomous task commands.

Example extension: a new cap color

This is a future implementation recipe, not an already-added feature.

First decide what the machine should do

Choose a color name and decide whether it replaces an existing color or needs a fifth destination. Reusing one existing bin for an initial experiment keeps the layout small. A fifth bin requires a validated new arrangement; this book does not propose an untested reachable coordinate.

Step	Future change and check
1. Give it an appearance	Add the exact color name to caps.colors with an RGB material value. Render sample caps and inspect the camera pixels.
2. Recognize it	Add the same name to perception.hue_ranges. Choose bands from measured HSV pixels, avoiding unintended overlap with existing bands.
3. Assign a destination	Add the same name to bins, or update the matching entries when replacing a color. The scene also uses the color entry for bin appearance.
4. Validate the path	Check approach, lift, transfer, release, and return with the actual new bin layout. Check the camera ROI and bin clearances too.
5. Test incrementally	Test saved images, then one live cap, then mixed colors and rejection cases. Confirm actual placement in the report.

Keep the names consistent

The coordinator uses the detected color string to look up its destination. A detector publishing “orange” with no “orange” bin entry will not produce a valid selected target. A bin entry without a matching material color can also break scene construction.

The message format already carries a string color, so adding a color does not normally require a new ROS message type. A new class can still require updated tests: the current synthetic examples and recorded campaign cover four colors.

Do not use cap names or simulator-assigned labels to classify the new color. Material configuration creates the appearance; the perception decision must still come from camera pixels.

Improve perception without rewiring everything

Keep the boundary clear: images in, useful visual detections out.

The present HSV method works well for separated, strongly colored caps against the belt. If lighting, markings, or object variety becomes more difficult, you can improve that stage while keeping the coordinator and motion action unchanged.

A staged path to a stronger detector

• Start with saved camera images and write down the cases the current method misses or accepts incorrectly.

• Improve lighting tolerance, shape filtering, or a learned detector using those examples.

• Publish compatible centres, radii, colors, confidence, frame IDs, and capture timestamps on the existing PixelCaps interface.

• Test projection and target rejection separately before allowing live robot motion.

A bounding box centre is not always the best suction point. If a future model detects partly hidden objects, choose a visible sealing surface or reject the instance. If it cannot provide a meaningful cap radius, update the downstream radius check deliberately instead of inventing a value to pass it.

When you need depth

The current projection assumes a known cap-top plane. Objects with different heights need additional information, such as calibrated depth, a second view, or an independently estimated height. That is a change to target generation as well as detection. Keep the output target expressed in base_link.

> **Keep evaluation separate**
> Synthetic labels can help build or score a future detector. At runtime, object IDs, true color labels, and simulator cap poses must remain outside the picking input path. Otherwise a successful demonstration would no longer establish image-based sorting.

After a detector change, retest latency as well as recognition. A correctly located but stale image can describe a cap that has already moved.

From simulation toward real hardware

The software boundaries are reusable; the physical assumptions need new evidence.

The current project commands simulated shoulder drives. It does not contain a driver for a physical Delta robot, PLC, motor bus, or real vacuum system. A hardware version needs an adapter that translates the same application intent into those devices and returns measured feedback.

Reusable idea	What a real installation must supply
MoveDelta goals and task sequence	A validated controller interface, measured joint motion, homing, and ownership of commands.
Joint-state feedback	Encoders, calibrated zero positions, correct directions, limits, and gear ratios.
RGB perception and projection	Camera driver, distortion correction, measured camera-to-robot calibration, and measured work surface.
Vacuum service and feedback	Valve control plus a pressure or seal signal, including lost-grasp detection.
Conveyor running service	Motor control, verified stopping behavior, and suitable feedback.
Stop/recovery policy	Hardware interlocks and a separately engineered machine-safety system.

Do not transfer simulated precision into a hardware claim

The model uses assumed masses, stabilized inertias, shoulder limits, and actuator gains. It omits gear backlash, link flexibility, detailed motor behavior, and vacuum pressure. Those choices make a useful functional simulator; they do not size real motors or prove real stopping distance.

Begin a future deployment with sensing and calibration, then restricted motion, then a single pick, before enabling repeated sorting. Keep the stop-and-pick strategy until the complete physical loop is understood. Continuous interception adds timing and tracking requirements on top of that baseline.

This implementation uses its own ROS motion action and adapter. A later ros2_control or other hardware-controller integration would be additional engineering, not a feature already configured here.

What has actually been validated

Recorded results establish a working baseline, with a clear limit on the claim.

Recorded run	Attempts	Picks	Correct bin
Initial single-cap cycle	1	1	1
Nominal repeated run	20	20	20
Injected failed vacuum	1	0	0
After moving-fastener fix	1	1	1

The nominal run placed **five caps of each color**, using varied positions and rotations with spawn seed 17. It published 3,969 RGB frames and ran about 396.8 simulation seconds. The recorded wall duration was about 597.9 seconds excluding startup; other tests ran concurrently, so that is not a dedicated speed benchmark.

Failure cases were exercised too

Check	Recorded outcome
Unreachable and NaN goals	Rejected before a new motion was commanded.
Empty scene and uncertain gray caps	Zero sorting attempts and zero grasp requests.
Failed vacuum	Two seal requests within one failed attempt; recovery lifted, returned home, and resumed feeding.
Stop and reset	Latched stop rejected belt/motion; reset allowed a later controlled move.
Focused automated tests	23 passed, covering geometry, paths, projection, colors, and rejection cases.

How to interpret 20 out of 20

It confirms this recorded seeded simulation campaign. It does not establish a production success rate, arbitrary-lighting robustness, real robot accuracy, or long-term reliability. The follow-up fastener check is a separate single cycle, not a rerun of the full 20-attempt campaign.

Validation date in the project report: 23 September 2026. This book reads that saved evidence; no new robot run was performed to produce the book. GUI startup/rendering was smoke-tested separately; autonomous campaigns ran headless.

Test a future change in small steps

A short feedback loop helps you locate the cause when something breaks.

Stage	What to check before proceeding
1. Offline checks	Run relevant vision/kinematics tests. Test changed inputs and a meaningful failure case.
2. Scene and geometry	Inspect dimensions, axes, attachments, collisions, and all moving parts through motion.
3. Camera and targets	Check a saved/live RGB image, pixels, TF, target coordinates, and target freshness.
4. One complete pick	Confirm physical attachment, lift, release, and correct measured placement.
5. Repeated mixed run	Use multiple colors and positions, then compare reports from more than one seed.
6. Recovery cases	Test empty/uncertain scenes, unreachable goals, failed grasp, and stop/reset again.

Useful existing commands

```
cd /home/amin24/orca/deltarobot
python3 -m pytest -q tests
DELTA_ATTEMPTS=1 DELTA_OUTPUT="$PWD/artifacts/my_single" \
  ./scripts/run_demo.sh --headless --max-time 100 \
  --exit-on-complete --capture-overview
```

The live negative-test suite is **scripts/validate_edges.sh**. It uses ROS domain 43 by default, with DELTA_TEST_DOMAIN as an override. It writes to artifacts/edge_tests/, so preserve the existing evidence before a future rerun. Its short test configurations differ from the ordinary 55-second empty-feed timeout.

> **Know which configuration a tool reads**
> scripts/sample_workspace.py and the existing test fixtures read the default source configuration. They do not automatically validate a file selected with DELTA_CONFIG. For a future custom layout, point your validation harness at that same file and test complete routes, not just isolated reachable points.

For each new experiment, record the configuration, seed, software version, attempted picks, successful lifts, correct placements, failures, and representative images. Do not count a requested release as a verified placement.

Find the first broken connection

Work from the camera toward the actuators instead of changing many settings at once.

Symptom	First checks
No ROS nodes or topics	Source Jazzy and the workspace; match ROS_DOMAIN_ID; inspect isaac.log for bridge startup.
Scene visible but robot idle	Opening USD alone does not launch sorting. Check /clock, /sorting/state, autostart, and a latched stop.
No camera frames	Check Isaac renderer logs, /camera/image_raw, and sensor-data QoS compatibility in the viewer.
Image looks right; no pixel detections	Inspect HSV/brightness thresholds, cap size, occlusion, overlap, borders, and contour confidence.
Pixels exist; no 3D targets	Check CameraInfo, image frame/timestamp, TF, the cap-top plane, picking ROI, and IK.
Targets exist; no selected pick	Check fresh post-settling images, color-to-bin mapping, confidence, and a feasible descent path.
Move rejected or timed out	Inspect action result and ROS logs; check full-path limits, clearance, and measured feedback.
Vacuum fails	Check stopped belt, physical cup position, cap uprightness, and calibration. Read the service result.
Reset refused	A cap may still be held or the cell stop may still be latched. Follow the order on page 20.
Part appears detached	Inspect moving-body ownership. Restart to load regenerated assets; a correct rest pose is insufficient.

Keep a useful problem report

Save the current configuration, state, service/action error, camera image, and run logs. A screenshot plus the first failing interface is much more informative than “the robot does not move.”

REFERENCE

A small working glossary

Enough terminology to read the files and discuss the machine.

Term	Simple meaning in this project
CAD assembly	Parts plus their relative placement; the starting point for the supplied robot.
Mesh	Triangles used to draw a part’s surface.
USD / USDC	The scene format; USDC is its compact binary form.
Rigid body	A simulated object that moves without bending.
Joint drive	The simulated actuator that tries to reach a commanded joint angle.
Closed chain	Linked bodies form a loop instead of a single branching tree.
Ball / spherical joint	A connection that allows rotation while keeping its two attachment points together.
TCP / tool0	The tool centre point; here the vacuum contact point.
Workspace	The region the tool may use, further limited by geometry and obstacles.
Singularity	A pose where the linkage loses a useful motion relationship or becomes poorly conditioned. The IK avoids nearby cases.
Extrinsics / TF	Where the camera or another frame is relative to the robot.
Intrinsics / CameraInfo	The lens/image model used to turn a pixel into a viewing ray.
HSV / contour	A color representation / the outline of an image blob.
QoS	Delivery settings that must be compatible between ROS publishers and subscribers.
Ground truth	Simulator-known facts used here for evaluation, not visual target selection.
Headless	Running the simulator without the graphical application window.

REFERENCE

Where to look next

Use this map to move from the book to the exact implementation or evidence.

Question	Project source
How do I start or install it?	README.md; scripts/build.sh; scripts/run_demo.sh
What messages connect the nodes?	docs/architecture.md; ros2_ws/src/delta_interfaces/
How was the CAD interpreted?	docs/robot_model.md; scripts/derive_robot.py; assets/generated/robot_geometry.json
How are image targets calculated?	docs/perception.md; delta_sorting/vision.py; projection_node.py
What sends joint commands?	delta_sorting/motion_node.py; kinematics.py; sim/ros_bridge.py
What chooses the next operation?	delta_sorting/coordinator_node.py
What changes the workcell?	sim/scene.py; delta_sorting/config/cell.yaml
Where are measured results?	docs/validation.md; artifacts/twenty_attempts/results.json; simulation.json
Where is the fastener correction evidence?	artifacts/floating_component_fix/verification.json and cycle/

In this table, delta_sorting/ Python modules are under ros2_ws/src/delta_sorting/delta_sorting/. Its config/ directory is ros2_ws/src/delta_sorting/config/. Short paths are used for readability.

A reliable way to keep learning

Choose one cap in a camera image, locate its pixel detection and projected target, then follow the state and action messages until it reaches a bin. Once that chain makes sense, each file becomes a place with a specific job rather than a large collection of unfamiliar code.

For a first addition, build a small observer or improve one measured failure case. Keep a reference run, make one change, and verify the full cycle. That gives you a working foundation for more ambitious perception, layout, and hardware work.

Book edition: 24 September 2026. Prepared from the local source, configuration, technical notes, and saved validation artifacts. Photographs are project simulation renders. Diagrams are explanatory schematics. Existing project files were preserved; the book and its companion documentation are new deliverables.